

מזהה סטודנט:

מזהה קורס: 20905 שם קורס: שפות תכנות

מספר שאלה	ניקוד מירבי	ציון
1	35.00	31.00
2	35.00	31.00
3	30.00	30.00

ציון בחינה סופי : 92.00

הבחינה הבדוקה בעמודים הבאים

87

מועד

20905

מספר הקורס

מספר תעודת זהות (9 ספרות)

לשימוש הבודק

① יורשים את השם קהלים הבונים:

lang.scm:

הגדרות the-grammar

המקום Proc-exp, המקום, (שג 5 :

(expression

~~(expression~~

("overload Proc

(orbn ["(separated-list identifier", ")"]

"→" expression)

overload-proc-exp)



המקום call-exp המקום:

(expression

(" (" expression (orbn-expression)))

call-exp)

data-structures & sem

קריאה

(define datatype Proc Proc?)

(procedure

(ids-list (list-of (list-of symbol)))

(bodies (list-of expression?))

(env environment?))

איזה קריאה היא? Proc? קריאה

התשובה



ציון שאלה 1:

עדכון דקדוק השפה -

הוספת דקדוק לביטוי overload-proc - 7/7

עדכון דקדוק call-exp - 3/3

מימוש:

עדכון מבנה נתונים של proc - 3/3

מימוש ביטוי overload-proc

מימוש תקין של בניית פרוצדורה והחזרת proc-val - 4/4

טיפול במצב של הגדרה ללא גופי פרוצדורה - 3/3

טיפול במצב של הגדרה עם מספר גופי פרוצדורה בעלות אותו מספר פרמטרים - 0/4

מימוש ביטוי call-exp

מימוש תקין של ביטוי call-exp ועדכון apply-procedure בהתאם - 8/8

טיפול במצב של הפעלה עם כמות פרמטרים שלא מתאימה לשום גוף פרוצדורה - 3/3

interf. scm דו"ח

(Value-of ~~proc~~ ^{value-of} ~~proc~~ ^{proc})
 (Value-of ₀₀₀
 (cases ₀₀₀
₀
₀
₀
 (overload-proc-exp (ids bodies) ^{(רשום כאן) ids} _←
 (begin (validate-proc-input _{ids} bodies)
 (Proc-ra (procedure ids bodies env))))
 (call-exp (rator rands)
 (let ((proc1 (exp1a1 → Proc (value-of rator env))))
 (args (map (lambda (exp) (value-of exp env)) rands)
) ~~close text~~
 (cases proc proc1
 (Procedure (list-ids bodies env)
 (search-and-apply list-ids bodies env args)))
 [א"כ 5 יום]

∴ value of σ_{dim} is 13.50 m for 20

```
(define (search-and-supply list-ids bodies env args)
```

~~if~~ (if (null? list-ids)) ✓

(expl: error "no procedure with this amount of arguments!" (length args))

```
(let ((ids (car list-ids)))
```

$Cif (= (\text{length } ids) (\text{length } args))$

(value-of (car bodies)
(extend-env ids args env))

```
(search-and-apply (cdr list-ids) (cdr bodies)
  env args)))
```

```
(define (validate-proc-input ids-list bodies)
```

(if (null? list-ids))

(error "You must define at least one Proc")
) #t ; dummy return, not used

אין טיפול במצב של הגדרה עם מספר גופי פרוצדורה בעלות אותו מספר פרמטרים

הסדר של הטקסט

(1) Search-and-apply להקשר ולמה של id
bodies וסדר ממש ואז אולי args

התבונן שם עליון של id וזה קטן מה id-lista של id
ואז קטן ממנה אולי id הטובת id מצינו
על אספק המאונכים שם מנסה id (לפי id)
המסמך ~~המסמך~~ ואולי (קטן id) שם
ה id של שם id, id מצינו מצינו error

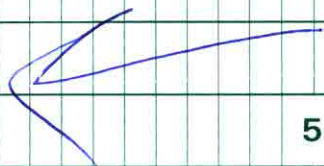
המסמך מ-extend-אולי קטן מה id של id
ואולי id, vals * שם id מה המסמך.
id מצינו, קטן id מ-extend-אולי מצינו

(2) validate-Proc-input - id מה id מה id

מצינו error מצינו מצינו id

מצינו מצינו מצינו קטן id מצינו מצינו

מצינו 2 מצינו מצינו



2. like

② מהן השאלות שצריך לנסות:

lang. sem: גרסה

the-grammar

(expression

("event create()")

event-create-exp)



(expression

("<" identifier ">" "+=" "}")

(separated-list expression ";") "}"

event-add-exp)



How data-structures look

(define datatype expr1 expr1?)

⋮



על מנת
לדעת מה

(event-val (first-of proc?))



⋮

הערה חשובה

(define expr1 → event

(lambda (v)

(cases expr1 v

(event-val (first-of procs) procs)

else (expr1-extractor 6 error 'reference v))))

דיווח interp. sem ~~המשפט~~

~~אחד~~ value-of ~~הערך~~

value-of 0 0 0

(cases 0 0 0

0

(event-create-exp ()



(event-val 'c)) ; return empty event

(event-add-exp (event, procs)



identifier זה event
שאמור לציין
צריך לפנות לסביבה
ולחפשו שם

(let ((old-procs (expr → env (value-of event env)))

(new-procs (map (lambda (exp)

צריך להפעיל גם מחלק
כדי לקבל אותם בצורת

(value-of exp env) procs)))

(begin (validate-add-input ^{proc} new-procs)

(let ((all-procs (combine old-procs new-procs)

(ref (apply-env env event)))

(setref! ref (event-val all-procs))))))

~~המשפט~~

(הערך של call-exp ~~הערך~~ 9

כעת נגדיר את add-event -> value of snow

(define (validate-add-input Procs)

(if (null? Procs)

(error "empty list not allowed for add events")

(check-all-procs Procs)))



(define (check-all-procs Procs)

(if (null? Procs)

#t

(cases exprval (car Procs)



(Proc-lai (Proc)

(check-all-procs (cdr Procs))))

(else (error "event can't contain
non-procedure elements")))))

(define (combine l1 l2)

(if (null? l2)

l1

(combine (cons l1 (car l2)) (cdr l2))))

יש את append לזה

value-of ~~env~~ ^{הערך} ~~env~~

call-exp

(value-of 0 0 0)

(cases 0 0 0)

(call-exp (rator rand))

(let ((v (value-of rator env))
(arg (value-of rand env))))

(cases expr1 v)

(Proc-Val (Proc)
(apply-Procedure Proc arg)))

(event-Val (Procs))

(if (null? Procs) (exp: error "can't invoke empty env")
(apply-multi-Proc Procs arg)))

(else (exp: error "only procedure or event can be invoked"))

~~: exp: error "value-of env"))))~~

(define

~~(apply-multi-Proc~~



: exp: error "value-of env"))))

(define (apply-multi-Proc Procs arg)

(let (res (apply-Procedure (car Procs) arg)))

(if (null? (cdr Procs))

res

(apply-multi-Proc 9 (cdr Procs) arg)))

הסדר של הפונקציות:

~~Express~~ 1. validate-add-input

לוקח event & Express
 קורא אם event סגור. event-add אין
 אחרת לא check-all-procs כדי

2. check-all-procs(procs)

לוקח event & Express
 לוקח proc ו- proc

אם event קטן event ו- proc
 אם event קטן event ו- proc
 אם event קטן event ו- proc

3. combine (1, 2)

לוקח 2 פונקציות / event ו- proc
 הסדר - האקלים ב 1 ו-2

פונקציות ש- event ו- proc

4. apply-multi-proc (event)

לוקח event & proc ו- event
 כל הפונקציות, event ו- proc
 הסדר שהם לחצות event ו- proc
 אלה הם event ו- proc 10 event
 (אם event קטן event ו- proc)

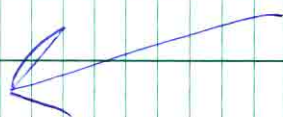
פירוס ציון
 שאלה 2-
 בסוף
 המחברת

468000

שאלה 3 - למי זה קרא



שאלה 3 - למי זה קרא



3

השורה ק - (5 val-num)



הסקר

מבנה הקטן של הסל - סופר, בעל
הסל, התבונן.

התבונן האשון - (המחבר) היא מחברת
שקלה מחברת אחרת והפסי אתה 6.

(הקטן)

התבונן חצי - קטן מחברת שקלה מסת X

אחריה 1-X.

ואכן

נראה מחברת 6 וקלה 6-1=5

~~אם~~ זה חצי קטן ו val-num ואכן

(5 val-num)

(הקטן) האשון (המחבר) מחברת מסת
ה let-exp מחברת אחרת מחברת X אחריה אחרת.



468000

ציון שאלה 2:

עדכון הדקדוק

ביטוי חדש – event-create 3/3

ביטוי חדש – event-add - 4/4

מימוש:

עדכון expval בסוג חדש - 2/2

ייצוג של event ע"י מבנה נתונים מתאים - 3/3

מימוש ביטוי event-create - בנייה נכונה והחזרת תשובה מסוג event-val - 4/4

מימוש ביטוי event-add

עדכון נכון של מבנה הנתונים המייצג את ה-event על פי חוקי העבודה בשפת וירמוז - 2/4

טיפול בשגיאות:

הוספה עם רשימה ריקה - 2/2

הוספה עם מרכיב שאינו פרוצדורה - 2/2

ניסיון לעדכן מישורו שלא מייצג event - 0/2

עדכון ביטוי call-exp ובעקבותיו apply-procedure

הפעלה נכונה של פרוצדורה ושל event - 6/6

טיפול בשגיאות:

ניסיון להפעיל משהו שאינו פרוצדורה או event - 2/2

ניסיון להפעיל evnet ריק - 1/1

100

Interp. scan - 92

(value-of 000

(cases 000

⋮

(event-create-exp c)

(event-val 'c)) ; return empty env-val

(event-add-exp (event, proc s)

(let ((~~old-procs~~ (expval → event (value-of event env))))

(~~new-procs~~ (map (lambda (exp)

(value-of exp env) proc s)))

(begin (validate-add-input new-procs)

(let* ((all-procs (combine ~~new-procs~~
old-procs new-procs)))

~~(new-env)~~

(ref (apply-env env event)))

(setref! ref (event-val all-procs))

(call-exp ...

(let x = proc(x) (x 6) in

x proc (x) (x n) 14)

468000

lang-sem

the-grammar

(expression

("event.create")

(event.create-exp)

(expression

("<identifier>" "+" ~~separated-list~~)

(separated-list expression ";") }

add event-add-exp(event, procs)

data-structures-sem

:define datatype-explains

o
o
o

(event-val (list-of Proc*))

:עזר עזר לזכור

(define explain-event

(lambda (v)

(cases explain v

(event-val (procs) procs)

else (explain-extractor-error 'reference-v)))

הערה

interp. sem

value-of

cases ...

...

(overload-proc-exp (ids bodies))

(begin (validate-proc ids bodies))

(proc-val (procedure ids bodies env)))

(call-exp (rator, rands))

cases

(search-and-apply rator rands

(let ((Proc1 (exp1 → proc (value-of rator env)))

(args (map (lambda (exp) (value-of exp env)) rands)))

(begin (validate-proc

(search-and-apply Proc

(cases Proc Proc1

(procedure (list-ids bodies env)

(search-and-apply list-ids bodies env args)

(define (search-and-apply list-ids bodies env args)

(let ((ids (car list-ids)))

(if (null? ids)

(error "no procedure with this amount of args found")

(if (= (length ids) (length args)

468000

car body env

לשימוש הבודק

רצ"ר

Proc-val $\rightarrow$ (value-of (car bodies))
(multi-extended ids arg env)

define datatype Proc Proc?

(Procedure
((list-of) Symbol?))

~~(overload-Proc-val (list-of Proc?))~~

(Procedure (ids
(ids (list-of (list-of Symbol?)))
(bodies (list-of expression?))
(env environment?))

define datatype over-Proc over-Proc?

~~over-Procedure~~
(ids (list-of (list-of Symbol?)))

arg. sim

(expression
(argno ("[" (separated-list identifier ",") "]" $\rightarrow$ expression)
overload-Proc-exp)

(expression

17

("[" expression "A" (argno expression) "]" call-exp)

גליון תשובות לשאלות רב-ברריות

הקף במעגל את התשובה שבחרת (לכל שאלה יש רק תשובה אחת נכונה).
אם תרצה לבטל תשובה שבחרת, סמן עליה X.
דוגמה לתשובה שבחרת: א ב ג ד ה ו ז ח ט
דוגמה לתשובה שבטלת: א ב ג ד ה ו ז ח ט

שאלה	תשובה	שאלה	תשובה
1	א ב ג ד ה ו ז ח ט	21	א ב ג ד ה ו ז ח ט
2	א ב ג ד ה ו ז ח ט	22	א ב ג ד ה ו ז ח ט
3	א ב ג ד ה ו ז ח ט	23	א ב ג ד ה ו ז ח ט
4	א ב ג ד ה ו ז ח ט	24	א ב ג ד ה ו ז ח ט
5	א ב ג ד ה ו ז ח ט	25	א ב ג ד ה ו ז ח ט
6	א ב ג ד ה ו ז ח ט	26	א ב ג ד ה ו ז ח ט
7	א ב ג ד ה ו ז ח ט	27	א ב ג ד ה ו ז ח ט
8	א ב ג ד ה ו ז ח ט	28	א ב ג ד ה ו ז ח ט
9	א ב ג ד ה ו ז ח ט	29	א ב ג ד ה ו ז ח ט
10	א ב ג ד ה ו ז ח ט	30	א ב ג ד ה ו ז ח ט
11	א ב ג ד ה ו ז ח ט	31	א ב ג ד ה ו ז ח ט
12	א ב ג ד ה ו ז ח ט	32	א ב ג ד ה ו ז ח ט
13	א ב ג ד ה ו ז ח ט	33	א ב ג ד ה ו ז ח ט
14	א ב ג ד ה ו ז ח ט	34	א ב ג ד ה ו ז ח ט
15	א ב ג ד ה ו ז ח ט	35	א ב ג ד ה ו ז ח ט
16	א ב ג ד ה ו ז ח ט	36	א ב ג ד ה ו ז ח ט
17	א ב ג ד ה ו ז ח ט	37	א ב ג ד ה ו ז ח ט
18	א ב ג ד ה ו ז ח ט	38	א ב ג ד ה ו ז ח ט
19	א ב ג ד ה ו ז ח ט	39	א ב ג ד ה ו ז ח ט
20	א ב ג ד ה ו ז ח ט	40	א ב ג ד ה ו ז ח ט

לשימוש פנימי

מספר התשובות הנכונות: _____ ציון: _____

שם הבודק: _____

468000